

Defend-The-Core

Infrastructure PME critique sécurisée

Guide d'installation pas à pas (KVM/QEMU)

Trace des étapes de déploiement de l'infrastructure en local

[KVM/QEMU & libvirt & OPNsense & NixOS & Zero Trust](#)

« Hôte Linux Mint 22, 32 Go RAM, 250 Go disque — 7 VMs segmentées en 4 VLANs. »

Table des matières

1. Introduction	4
1.1. Pré-requis matériels	4
1.2. Système d'exploitation hôte	4
2. Vérification du support KVM	4
2.1. Flags CPU vmx/svm	5
2.2. Périphérique /dev/kvm	5
2.3. Récapitulatif matériel	5
3. Installation de KVM/QEMU	5
3.1. Rôle de chaque paquet	5
4. Configuration post-installation	6
4.1. Activation du service libvirtd	6
4.2. Groupes libvirt et kvm	6
4.3. Création du pool de stockage par défaut	6
4.4. Vérifications	7
5. Plan d'allocation des ressources	7
6. Création des réseaux isolés (VLANs)	7
6.1. Les quatre réseaux à créer	8
6.2. Format XML d'un réseau isolé	8
6.3. Définition et démarrage	8
7. Téléchargement des ISO	8
7.1. Stockage des ISO	9
8. Création de la VM OPNsense	9
8.1. Commande virt-install	9
8.2. Paramètres clés	9
8.3. Configuration post-installation	10
9. Création de la VM Wazuh	10
9.1. Commande virt-install	11
9.2. Installation Ubuntu de base	11
9.3. Application du script d'installation	11
10. Création du Bastion NixOS	11

10.1. Commande virt-install	11
10.2. Installation de NixOS	11
10.3. Configuration déclarative du dépôt	12
11. Création des serveurs et postes clients	12
11.1. Ubuntu Server (VLAN 30)	12
11.2. Windows Server (VLAN 30)	13
11.3. Windows 10 et Kali (VLAN 10)	13
12. Vérification et tests	13
12.1. Connectivité et DHCP	13
12.2. Tests inter-VLAN (zero trust)	14
12.3. Scénarios de validation	14
13. Commandes utiles au quotidien	14
14. Dépannage	15

1. Introduction

Ce document est la **trace pas à pas** de l'installation de l'infrastructure Defend-The-Core sur un poste de travail local, à l'aide de l'hyperviseur **KVM/QEMU** piloté par **libvirt**. L'objectif est double : constituer un journal reproductible de toutes les opérations effectuées (pré-requis, installation des paquets, configuration des réseaux isolés, création des VMs, tests) et fournir un mode opératoire utilisable pour rejouer le déploiement à l'identique sur une autre machine.

Contrairement à VirtualBox, KVM/QEMU est l'hyperviseur **natif du noyau Linux** : il bénéficie de l'accélération matérielle directe (VT-x/AMD-V), d'une intégration profonde avec systemd et d'un outillage mature (libvirt, virt-manager). Ce choix reflète une démarche d'ingénierie réaliste : sur un serveur Linux de production, KVM/QEMU est la solution de virtualisation de référence. Les réseaux internes VirtualBox sont ici remplacés par des **réseaux isolés libvirt**, qui simulent le zoning 802.1Q sans équipement actif.

Chaque section décrit une étape avec ses commandes exactes (exécutées en tant que simple utilisateur membre des groupes **libvirt** et **kvm**, sauf mention de **sudo**) et ses vérifications. Les scripts d'automatisation du dépôt sont appliqués après installation manuelle du système d'exploitation de chaque VM.

1.1. Pré-requis matériels

L'infrastructure fait tourner jusqu'à 7 VMs simultanément. Les ressources matérielles de l'hôte doivent couvrir la somme des allocations, la marge de l'hyperviseur et l'OS hôte lui-même.

Tableau 1 — Pré-requis matériels de l'hôte

Ressource	Minimum	Justification
RAM	32 Go	Somme des VMs (21 Go) + OS hôte (5 Go) + marge
CPU	8 cœurs, VT-x/AMD-V	4 vCPU Wazuh + 2 OPNsense + autres ; virtualisation matérielle obligatoire
Disque	250 Go	ISOs (~20 Go) + disques VMs (~200 Go) + snapshots
Réseau	1 carte Ethernet	Accès WAN pour téléchargements / mise à jour

1.2. Système d'exploitation hôte

L'hôte est un **Linux Mint 22 (Wilma)**, basé sur Ubuntu 24.04 LTS. Ce choix garantit un accès direct aux paquets Debian/Ubuntu de KVM/QEMU et libvirt, une longue durée de support et un environnement de bureau familier pour lancer **virt-manager**. Toutes les commandes de ce guide sont valables sur toute distribution Debian/Ubuntu récente.

Vérifier la version de l'hôte avant toute chose :

```
lsb_release -a
# Description: Linux Mint 22 Wilma / Ubuntu 24.04 base
```

2. Vérification du support KVM

KVM nécessite l'**extension de virtualisation matérielle** du processeur : Intel VT-x (flag `vmx`) ou AMD-V (flag `svm`). Sans elle, QEMU bascule en émulation logicielle et les VMs sont inutilisables. Cette vérification est le préalable non négociable.

2.1. Flags CPU `vmx/svm`

On cherche la présence d'un des deux flags dans `/proc/cpuinfo`. L'awk ci-dessous renvoie un verdict explicite :

```
awk '/vmx|svm/{found=1} END{print found?"vmx/svm detecte OK":"NON detecte X"}' /proc/cpuinfo
```

2.2. Périphérique `/dev/kvm`

Si le module noyau `kvm` (et `kvm_intel` ou `kvm_amd`) est chargé, le périphérique caractère `/dev/kvm` existe :

```
ls -la /dev/kvm
# crw-rw----+ 1 root kvm ... /dev/kvm
```

2.3. Récapitulatif matériel

Une vue d'ensemble de l'architecture CPU, du modèle et de la mémoire disponible permet de valider que l'hôte est dimensionné :

```
lscpu | awk '/^Architecture|^CPU\(|^Model name|^Vendor/{print}'
free -h
```

Sortie attendue (exemple) : Architecture `x86_64`, Vendor ID `GenuineIntel`, 8 cœurs, Mem total `~32 Gi`. Si `vmx/svm` n'est pas détecté alors que le CPU le supporte, il faut activer la virtualisation dans le BIOS/UEFI (option « Intel VT-x » ou « SVM Mode »).

3. Installation de KVM/QEMU

La pile logicielle se compose de l'hyperviseur **QEMU/KVM**, de l'API de gestion **libvirt**, des clients en ligne de commande et graphique, et d'utilitaires de découverte d'OS. L'installation se fait en une seule commande depuis les dépôts officiels de Linux Mint/Ubuntu.

```
sudo apt update && \
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients \
bridge-utils virt-manager virtinst libosinfo-bin
```

3.1. Rôle de chaque paquet

Chaque paquet a un rôle précis dans la chaîne de virtualisation. Comprendre ces rôles facilite le débannage ultérieur.

Tableau 2 — Paquets installés et leur rôle

Paquet	Rôle
--------	------

qemu-kvm	Hyperviseur utilisateur : émulation des périphériques + accès à /dev/kvm pour l'accélération matérielle
libvirt-daemon-system	Service libvirtd : API de gestion des VMs, réseaux et pools de stockage
libvirt-clients	Clients CLI virsh / virt-xml pour piloter libvirtd
bridge-utils	Outil brctl : création de ponts réseau (bridges) pour le mode ponté
virt-manager	Interface graphique GTK pour créer et gérer les VMs (utile pour l'installation interactive)
virtinst	Commande virt-install : création de VMs en ligne de commande (scriptable)
libosinfo-bin	Base osinfo-db : détection automatique de l'OS pour optimiser le type de VM (os-variant)

4. Configuration post-installation

Une fois les paquets installés, trois actions restent obligatoires : activer le service `libvirtd`, accorder à l'utilisateur les droits d'accès à la virtualisation, et créer le pool de stockage par défaut qui contiendra les disques des VMs.

4.1. Activation du service libvirtd

Le service doit démarrer maintenant et à chaque boot :

```
sudo systemctl enable --now libvirtd
```

Vérifier l'état (active (running)) :

```
systemctl is-active libvirtd
```

4.2. Groupes libvirt et kvm

Par défaut, seul root peut piloter libvirt. On ajoute l'utilisateur courant aux groupes `libvirt` (gestion des VMs) et `kvm` (accès à /dev/kvm), puis on recharge les groupes de la session courante :

```
sudo usermod -aG libvirt,kvm $USER
newgrp libvirt
```

Après `newgrp`, les commandes `virsh` s'exécutent sans `sudo`. Pour une prise d'effet complète (toutes les sessions), une déconnexion/reconnexion de l'utilisateur est plus sûre.

4.3. Création du pool de stockage par défaut

Le pool `default` est un répertoire, `/var/lib/libvirt/images`, qui accueillera les fichiers `.qcow2` des disques VMs. On le crée, on en définit le propriétaire (l'utilisateur système `libvirt-qemu`), puis on le déclare, le construit, le démarre et l'active au boot :

```
sudo mkdir -p /var/lib/libvirt/images && \
sudo chown libvirt-qemu:kvm /var/lib/libvirt/images && \
virsh pool-define-as default --type dir --target /var/lib/libvirt/images && \
virsh pool-build default && \
virsh pool-start default && \
virsh pool-autostart default
```

4.4. Vérifications

Trois vérifications confirment que l'environnement est opérationnel : le réseau NAT par défaut existe, le pool de stockage est actif et ses informations sont cohérentes.

```
virsh net-list --all
virsh pool-list --all
virsh pool-info default
```

Le réseau **default** (virbr0, 192.168.122.0/24, mode NAT) sert de **WAN** pour OPNsense : il donne accès à Internet via l'hôte. Il doit apparaître comme **active** et **autostart** ; si ce n'est pas le cas : `virsh net-start default && virsh net-autostart default`.

5. Plan d'allocation des ressources

Le dimensionnement reflète le rôle de chaque VM. Les VMs « cœur » (OPNsense, Wazuh, Bastion, Ubuntu Server) tournent en permanence ; les VMs Windows et Kali ne sont démarrées qu'à la demande, ce qui permet de dépasser la RAM physique si toutes ne sont pas allumées simultanément. Chaque VM est rattachée à un (ou plusieurs) VLAN(s) via son interface réseau.

Tableau 3 — Plan d'allocation des ressources (7 VMs)

VM	RAM	vCPU	Disque	Priorité	VLAN
OPNsense	2 Go	2	20 Go	Cœur	tous (WAN+10/20/30/99)
Wazuh	8 Go	4	50 Go	Cœur	99
Bastion NixOS	1 Go	1	10 Go	Cœur	99
Ubuntu Server	2 Go	2	20 Go	Cœur	30
Windows Server	4 Go	2	40 Go	À la demande	30
Windows 10	2,5 Go	2	40 Go	À la demande	10
Kali	1,5 Go	2	20 Go	À la demande	10

Bilan mémoire. Somme des RAM VMs = 21 Go. En comptant l'OS hôte (~5 Go) et la marge de libvirt/QEMU, on consomme environ **26 Go sur 32 Go** lorsque toutes les VMs « cœur » tournent. Les VMs à la demande (Windows Server, Windows 10, Kali) ne sont démarrées qu'une à la fois lors des démonstrations, ce qui évite la sur-allocation.

6. Création des réseaux isolés (VLANs)

Le zoning ANSSI est implémenté par **quatre réseaux libvirt isolés**, un par VLAN. En mode **isolated**, le réseau n'a **ni NAT ni passerelle par défaut vers l'extérieur** : les VMs ne peuvent communiquer qu'entre elles sur ce segment. Le DHCP n'est pas géré par libvirt — c'est **OPNsense qui distribue les baux** sur chaque VLAN via ses interfaces LAN/OPT. Chaque réseau porte l'adresse de sous-réseau correspondante, avec OPNsense en .1.

6.1. Les quatre réseaux à créer

Tableau 4 — Réseaux isolés libvirt (un par VLAN)

Réseau libvirt	VLAN	Plage d'adresses	Rôle de la zone
vlan10	10	10.10.10.0/24	Bureautique (postes, vecteur d'attaque)
vlan20	20	10.10.20.0/24	DMZ (services exposés, proxy/VPN)
vlan30	30	10.10.30.0/24	Critique (serveurs métier, BDD, AD)
vlan99	99	10.10.99.0/24	Admin/SIEM (Wazuh, bastion)

6.2. Format XML d'un réseau isolé

Chaque réseau est décrit dans un fichier XML. Ci-dessous le modèle pour `vlan10.xml` — les trois autres s'en déduisent en adaptant le nom, l'UUID (optionnel) et la plage d'adresses :

```
<network>
  <name>vlan10</name>
  <!-- mode isolated : pas de forward, donc ni NAT ni route externe -->
  <bridge name='virbr10' stp='off' delay='0' />
  <domain name='vlan10' />
  <!-- Pas de <ip> ni de <dhcp> : OPNsense gere le DHCP sur ce segment -->
</network>
```

Explication des éléments : `<name>` identifie le réseau côté libvirt ; l'**absence de balise `<forward>`** est ce qui active le mode isolé (aucune sortie vers l'hôte ou Internet) ; `<bridge>` crée le pont Linux `virbr10` ; on omet volontairement `<ip>` et `<dhcp>` pour laisser OPNsense gérer l'adressage et la distribution des baux.

6.3. Définition et démarrage

Pour chaque réseau (répéter pour `vlan10`, `vlan20`, `vlan30`, `vlan99`) :

```
virsh net-define vlan10.xml
virsh net-start vlan10
virsh net-autostart vlan10
```

Vérification finale — les 5 réseaux (default + 4 VLANs) doivent être actifs :

```
virsh net-list --all
# Name      State    Autostart  Persistent
# default   active   yes        yes
# vlan10     active   yes        yes
# vlan20     active   yes        yes
# vlan30     active   yes        yes
# vlan99     active   yes        yes
```

7. Téléchargement des ISO

Chaque VM s'installe depuis une image ISO. On les regroupe dans un sous-répertoire `/var/lib/libvirt/images/iso/` du pool de stockage, afin que `virt-manager` et `virt-install` les trouvent au même endroit.

Tableau 5 — Images ISO requises

VM	OS	Source de téléchargement	Taille
OPNsense	OPNsense (amd64 DVD)	https://opnsense.org/download/	~1,2 Go
Wazuh / Ubuntu	Ubuntu Server 22.04 LTS	https://releases.ubuntu.com/22.04/	~2,0 Go
Bastion	NixOS (minimal ISO)	https://nixos.org/download/	~0,8 Go
Windows Server	Windows Server 2022 (éval.)	Portail d'évaluation Microsoft	~5,0 Go
Windows 10	Windows 10 (ISO)	Outil de création Microsoft	~5,0 Go
Kali	Kali Linux (installer)	https://www.kali.org/get-kali/	~4,0 Go

7.1. Stockage des ISO

Créer le répertoire dédié, puis y télécharger (ou y copier) les ISO :

```
sudo mkdir -p /var/lib/libvirt/images/iso
sudo chown libvirt-qemu:kvm /var/lib/libvirt/images/iso
cd /var/lib/libvirt/images/iso
# Exemple : ISO Ubuntu Server 22.04
sudo wget https://releases.ubuntu.com/22.04/ubuntu-22.04.X-live-server-amd64.iso
# Renommer pour gagner en lisibilité :
# opnsense-dvd.iso, ubuntu-22.04.iso, nixos.iso, win2022.iso, win10.iso, kali.iso
```

Conserver les noms courts indiqués en commentaire simplifie les commandes `virt-install` des sections suivantes.

8. Création de la VM OPNsense

OPNsense est le **pare-feu/routeur** central : il possède 5 interfaces réseau, une par segment (WAN + 4 VLANs). C'est la première VM à créer, car toutes les autres dépendent de son routage et de son DHCP.

8.1. Commande virt-install

On crée la VM avec `virt-install`, en lui attachant les 5 réseaux dans l'ordre WAN puis VLANs. Le disque est en `qcow2` ; l'ISO OPNsense est montée comme CD-ROM pour l'installation.

```
virt-install \
  --name opnsense \
  --memory 2048 --vcpus 2 \
  --disk path=/var/lib/libvirt/images/opnsense.qcow2,size=20,bus=virtio \
  --cdrom /var/lib/libvirt/images/iso/opnsense-dvd.iso \
  --network network=default \
  --network network=vlan10 \
  --network network=vlan20 \
  --network network=vlan30 \
  --network network=vlan99 \
  --os-variant freebsd13.0 \
  --graphics spice \
  --noautocons
```

8.2. Paramètres clés

Tableau 6 — Paramètres de virt-install pour OPNsense

Paramètre	Valeur	Rôle
--name	opnsense	Identifiant libvirt de la VM
--memory	2048	2 Go de RAM (pare-feu peu gourmand)
--vcpus	2	2 cœurs virtuels pour le filtrage et l'IDS
--disk	opnsense.qcow2, 20 Go, virtio	Disque principal en qcow2, bus virtio (performances)
--cdrom	opnsense-dvd.iso	ISO d'installation démarrée au premier boot
--network (x5)	default, vlan10-30, vlan99	WAN (NAT) + une interface par VLAN isolé
--os-variant	freebsd13.0	Optimise le type de machine (OPNsense est basé sur FreeBSD)

8.3. Configuration post-installation

Au premier démarrage, OPNsense attribue ses interfaces à la console. L'ordre des `--network` ci-dessus correspond à l'ordre d'énumération vu par FreeBSD. On assigne :

Tableau 7 — Affectation des interfaces OPNsense

Interface OPNsense	Réseau libvirt	Rôle	Adresse
WAN	default (virbr0)	Accès Internet via NAT hôte	DHCP (192.168.122.x)
LAN	vlan10	Bureautique + passerelle du VLAN 10	10.10.10.1/24
OPT1	vlan20	DMZ	10.10.20.1/24
OPT2	vlan30	Zone critique	10.10.30.1/24
OPT3	vlan99	Admin/SIEM	10.10.99.1/24

On active ensuite le DHCP sur chaque interface LAN/OPT (Services > DHCPv4), puis on applique les scripts du dépôt qui automatisent la suite : création des règles de pare-feu, du NAT, et de l'export syslog vers Wazuh.

```
# Depuis l'hôte, pousser puis appliquer les scripts OPNsense du depot
cd ~/defend-the-core/opnsense/scripts
# 00-initial-setup.sh -> configuration de base (mot de passe, DNS)
# 01-vlan-interfaces.sh -> déclenchement des interfaces VLAN
# 02-firewall-rules.sh -> regles default-deny inter-VLAN
# 03-nat-gateway.sh -> NAT sortant et port-forwarding
```

9. Création de la VM Wazuh

Wazuh est le **SIEM/XDR** du VLAN 99. C'est la VM la plus gourmande (8 Go, 4 vCPU) car elle héberge le Manager, l'Indexer OpenSearch et le Dashboard. On l'installe sur Ubuntu Server 22.04, puis on applique le script d'installation du dépôt qui déploie les trois composants en une passe.

9.1. Commande virt-install

```
virt-install \
  --name wazuh \
  --memory 8192 --vcpus 4 \
  --disk path=/var/lib/libvirt/images/wazuh.qcow2,size=50,bus=virtio \
  --cdrom /var/lib/libvirt/images/iso/ubuntu-22.04.iso \
  --network network=vlan99 \
  --os-variant ubuntu22.04 \
  --graphics spice
```

Une seule interface sur `vlan99` : Wazuh est en zone admin, il n'a pas vocation à être exposé. On lui attribue une IP fixe (10.10.99.10) pendant l'installation Ubuntu.

9.2. Installation Ubuntu de base

Installation minimale Ubuntu Server 22.04 : créer un utilisateur `wazuh`, configurer l'adresse statique **10.10.99.10/24**, passerelle **10.10.99.1** (OPNsense) et SSH activé. Aucun paquet supplémentaire n'est sélectionné (pas de snap Docker, etc.) — le script du dépôt se charge de tout.

9.3. Application du script d'installation

Une fois Ubuntu installé et la VM redémarrée, on pousse le script du dépôt et on l'exécute. Il ajoute le dépôt APT officiel Wazuh, installe Manager/Indexer/Dashboard et génère les mots de passe.

```
# Sur la VM Wazuh, apres transfert du depot
cd ~/defend-the-core/wazuh/install
sudo bash install-wazuh-manager.sh

# Verification : les 3 services doivent etre 'active (running)'
systemctl status wazuh-manager wazuh-indexer wazuh-dashboard
```

Le dashboard est ensuite accessible (depuis le bastion ou via un tunnel SSH) sur **https://10.10.99.10**. La configuration de l'écoute syslog pour OPNsense et le déploiement des agents sont détaillés dans le document Annexe 2 (Sécurité & Wazuh).

10. Création du Bastion NixOS

Le bastion est le **Privileged Access Workstation (PAW)** du VLAN 99. Léger (1 Go, 1 vCPU), il tire sa force de l'**immuabilité déclarative** de NixOS : toute la configuration système vit dans `configuration.nix`, et toute modification manuelle est écrasée au prochain `nixos-rebuild switch`.

10.1. Commande virt-install

```
virt-install \
  --name bastion \
  --memory 1024 --vcpus 1 \
  --disk path=/var/lib/libvirt/images/bastion.qcow2,size=10,bus=virtio \
  --cdrom /var/lib/libvirt/images/iso/nixos.iso \
  --network network=vlan99 \
  --os-variant nixos-22.05 \
  --graphics spice
```

10.2. Installation de NixOS

Démarrer sur l'ISO, puis lancer l'installation depuis la console live. On partitionne le disque, on génère la configuration initiale, puis on la remplace par celle du dépôt avant le rebuild.

```
# Console live NixOS
sudo su -
parted /dev/vda -- mklabel gpt
parted /dev/vda -- mkpart primary 1MiB -8GiB
parted /dev/vda -- mkpart primary linux-swap -8GiB 100%
mkfs.ext4 -L nixos /dev/vda1
mount /dev/disk/by-label/nixos /mnt
nixos-generate-config --root /mnt
```

10.3. Configuration déclarative du dépôt

Le dépôt fournit deux fichiers : `configuration.nix` (configuration système : sshd en clés uniquement, ProxyJump, pare-feu, durcissement) et `hardware-configuration.nix` (modules matériels et points de montage). On les copie en écrasant la configuration générée, puis on reconstruit le système :

```
# Copie des fichiers de configuration du depot
cp ~/defend-the-core/bastion-nixos/configuration.nix /mnt/etc/nixos/
cp ~/defend-the-core/bastion-nixos/hardware-configuration.nix /mnt/etc/nixos/

# Reconstruction et activation (le systeme devient immuable)
nixos-install
reboot

# Apres redemarrage, toute modification ulterieure se fait par :
sudo nixos-rebuild switch
```

À partir de là, le bastion (10.10.99.20) est le **seul** hôte autorisé par OPNsense à joindre le port 22 des serveurs du VLAN 30. Toute administration passe par `ssh -J` (ProxyJump) depuis le bastion.

11. Création des serveurs et postes clients

Les quatre VMs restantes (Ubuntu Server, Windows Server, Windows 10, Kali) sont créées sur le même modèle : `virt-install` avec une interface sur le VLAN adapté, installation manuelle de l'OS, puis application des scripts d'automatisation du dépôt. Le tableau ci-dessous récapitule le rattachement réseau et le script d'installation de chacune.

Tableau 8 — Serveurs et postes clients

VM	OS	VLAN	Script(s) d'installation du dépôt
Ubuntu Server	Ubuntu 22.04	30	ubuntu-server/scripts/00-06 (base, web stack, harden SSH, agent Wazuh, auditd, UFW, CIS)
Windows Server	Windows Server 2022	30	windows-server/scripts (Install-ADForest, Configure-DNS-DHCP, Harden, Configure-GPO)
Windows 10	Windows 10	10	windows-client/scripts/Harden-Win10.ps1 (+ Install-WazuhAgent)
Kali	Kali Linux	10	kali-attacker/scripts/setup-kali.sh (outils + scénarios d'attaque)

11.1. Ubuntu Server (VLAN 30)

```

virt-install \
  --name ubuntu-server \
  --memory 2048 --vcpus 2 \
  --disk path=/var/lib/libvirt/images/ubuntu-server.qcow2,size=20,bus=virtio \
  --cdrom /var/lib/libvirt/images/iso/ubuntu-22.04.iso \
  --network network=vlan30 \
  --os-variant ubuntu22.04 --graphics spice

```

Après installation (IP fixe 10.10.30.10), on enchaîne les scripts numérotés du dépôt : **00-base-setup.sh** à **06-cis-hardening.sh** configurent la pile Web (Nginx + PostgreSQL), durcissent SSH, installent l'agent Wazuh, configurent auditd, UFW et appliquent le CIS Benchmark.

11.2. Windows Server (VLAN 30)

```

virt-install \
  --name win-server \
  --memory 4096 --vcpus 2 \
  --disk path=/var/lib/libvirt/images/win-server.qcow2,size=40,bus=virtio \
  --cdrom /var/lib/libvirt/images/iso/win2022.iso \
  --network network=vlan30 \
  --os-variant win2k22 --graphics spice

```

Séquence PowerShell après installation (IP 10.10.30.20) : **Install-ADForest.ps1** (forêt Active Directory), **Configure-DNS-DHCP.ps1**, **Harden-WindowsServer.ps1** (SMBv1 off, Defender, NLA) puis **Configure-GPO.ps1** (politique de mots de passe, restrictions logicielles).

11.3. Windows 10 et Kali (VLAN 10)

```

virt-install \
  --name win10 \
  --memory 2560 --vcpus 2 \
  --disk path=/var/lib/libvirt/images/win10.qcow2,size=40,bus=virtio \
  --cdrom /var/lib/libvirt/images/iso/win10.iso \
  --network network=vlan10 \
  --os-variant win10 --graphics spice

```

Windows 10 (10.10.10.50) reçoit **Harden-Win10.ps1** et l'agent Wazuh ; il simule le poste employé (vecteur d'attaque). Kali (10.10.10.100) est provisionné via **setup-kali.sh** qui installe les outils et les trois scénarios d'attaque (recon nmap, bruteforce SSH, mouvement latéral) utilisés pour les démonstrations.

12. Vérification et tests

Une fois les VMs déployées, on valide que le zoning et la détection fonctionnent. Les tests couvrent la connectivité réseau, le bon fonctionnement du DHCP par OPNsense, et les trois scénarios de démonstration du projet.

12.1. Connectivité et DHCP

OPNsense distribue les baux sur chaque VLAN. On les consulte depuis l'hôte :

```

virsh net-dhcp-leases vlan10
# Doit afficher les baux distribués par OPNsense (10.10.10.x)

```

Diagnostic d'une VM individuelle — son adresse MAC/IP et son état :

```

virsh list --all
virsh domifaddr ubuntu-server

```

12.2. Tests inter-VLAN (zero trust)

Le cœur du contrôle : un poste du VLAN 10 ne doit **pas** atteindre les VLANs 30/99. Depuis Win10 (10.10.10.50), un ping vers 10.10.30.10 doit échouer — c'est OPNsense qui droppe le flux (default-deny).

```
# Depuis Win10 (VLAN 10) : doit ECHOUER (OPNsense bloque)
ping 10.10.30.10 # -> Delai d'attente dépasse
ping 10.10.99.10 # -> idem, zone admin isolée
```

L'administration, elle, doit fonctionner via le bastion (ProxyJump) :

```
# Depuis le bastion (10.10.99.20), SSH vers le serveur (VLAN 30) :
ssh -J admin@10.10.99.20 admin@10.10.30.10
# Fonctionne : OPNsense autorise le bastion -> VLAN 30:22
```

12.3. Scénarios de validation

Tableau 9 — Scénarios de validation

#	Scénario	Action	Résultat attendu
1	Détection SIEM	Bruteforce SSH depuis Kali vers Ubuntu Server	Wazuh détecte > active response bloque l'IP automatiquement
2	Zero Trust	Ping Win10 > VLAN 30 / VLAN 99	Échec : OPNsense bloque (default-deny), logs de drop visibles
3	PAW NixOS	SSH direct vs ProxyJump via bastion	Direct impossible ; bastion > serveur fonctionne ; immuabilité vérifiée

13. Commandes utiles au quotidien

Aide-mémoire des commandes `virsh` et `virt-manager` les plus utilisées pour piloter l'infrastructure au quotidien.

Tableau 10 — Cheat sheet `virsh` / `virt-manager`

Commande	Description
<code>virsh list --all</code>	Liste toutes les VMs (allumées et éteintes)
<code>virsh start <vm></code>	Démarre une VM éteinte
<code>virsh shutdown <vm></code>	Arrêt propre (signal ACPI) de la VM
<code>virsh destroy <vm></code>	Arrêt forcé (équivalent débrancher) — à éviter
<code>virsh reboot <vm></code>	Redémarrage propre d'une VM
<code>virsh console <vm></code>	Console série (Ctrl+] pour quitter)
<code>virsh net-list --all</code>	Liste tous les réseaux libvirt
<code>virsh net-start <net></code>	Démarre un réseau (ex. vlan10)
<code>virsh domifaddr <vm></code>	Adresses MAC/IP d'une VM

virsh net-dhcp-leases <net>	Baux DHCP actifs sur un réseau
virsh pool-list --all	Liste les pools de stockage
virsh snapshot-create-as <vm> nom	Crée un instantané d'une VM
virsh snapshot-list <vm>	Liste les instantanés d'une VM
virsh snapshot-revert <vm> nom	Restaure une VM sur un instantané
virt-manager	Ouvre l'interface graphique de gestion des VMs
virt-viewer <vm>	Ouvre la console graphique d'une VM

Astuce : avant toute manipulation risquée (mise à jour, test d'attaque), créer un instantané avec **virsh snapshot - create - as** permet de revenir en arrière instantanément.

14. Dépannage

Tableau récapitulatif des problèmes les plus fréquents rencontrés pendant l'installation, leur cause probable et la résolution.

Tableau 11 — Problèmes courants et solutions

Problème	Cause probable	Solution
La VM ne démarre pas	Disque manquant ou chemin ISO incorrect	Vérifier virsh pool-info default ; chemin .qcow2 et .iso absolus dans virt-install
« permission denied » sur /dev/kvm	Utilisateur hors du groupe kvm	sudo usermod -aG kvm \$USER ; newgrp kvm (ou déconnexion/reconnexion)
virsh : « impossible de se connecter à qemu »	libvirtd inactif ou ACL libvirt manquante	sudo systemctl enable --now libvirtd ; ajouter l'utilisateur au groupe libvirt
Pas d'accès réseau dans une VM	Réseau libvirt non démarré ou VM sur le mauvais réseau	virsh net-list --all ; virsh net-start vlan10 ; vérifier --network dans virt-install
Pas de bail DHCP sur un VLAN	DHCP libvirt désactivé (isolated) & OPNsense non configuré	Normal : OPNsense gère le DHCP. Configurer Services > DHCPv4 sur l'interface OPNsense
Ping inter-VLAN réussit alors qu'il devrait échouer	Règle default-deny absente ou mal ordonnée dans OPNsense	Ajouter une règle block any any en fin de liste sur l'interface ; vérifier l'ordre
Wazuh ne reçoit pas les logs OPNsense	Syslog non configuré ou port 514/udp fermé	Configurer ossec.conf (remote syslog 514/udp) & règle d'export OPNsense vers 10.10.99.10
L'interface graphique virt-manager est lente	Spice sans accel. 3D sur VM Windows	Activer « Affichage Spice + GL » ou utiliser virt-viewer ; vérifier les drivers virtio
La VM Kali est injoignable après installation	IP non fixée ou sur le mauvais VLAN	Assigner 10.10.10.100/24 via NetworkManager ; gw 10.10.10.1 (OPNsense)

Fin du guide d'installation. Les configurations détaillées de chaque composant (OPNsense, Wazuh, durcissement, bastion NixOS) font l'objet des annexes 1 à 4.